AWS — 3–4 YOE Scenario-Based Troubleshooting Q&A

1. EC2 instance is running, but you cannot connect to it through SSH. What will you check?
Answer:
First, I verify that the EC2 instance is in the running state and check its system and instance status checks. Then I verify the Security Group, Network ACL, route table, subnet, and whether the instance has the expected public or private connectivity.
I also check whether SSH is running on the instance and whether the correct key pair and username are being used.
After identifying the network or SSH issue, I fix it and verify SSH connectivity again.
Simple ga gurthupettuko:
EC2 running unna SSH avvakapothe → instance status check cheyyi → Security Group/NACL/route check cheyyi → SSH service and key verify cheyyi → fix chesi connect avvu.
Eng: EC2 is running but SSH fails → check instance health, Security Group, NACL, routing, SSH service, username, and key → fix and reconnect.
Example:
aws ec2 describe-instances --instance-ids <instance-id> → Instance state, subnet, private IP, and public IP chudu.
aws ec2 describe-instance-status --instance-ids <instance-id> → System and instance status checks verify cheyyi.
aws ec2 describe-security-groups --group-ids <sg-id> → SSH port 22 inbound rule check cheyyi.
aws ec2 describe-route-tables → Subnet route Internet Gateway/NAT ki correct ga undho chudu.
ssh -i <key.pem> ec2-user@<public-ip> → SSH connectivity test cheyyi.
sudo systemctl status sshd → EC2 lo SSH service running lo undho check cheyyi.
Issue Security Group/route/SSH service ayithe → fix chesi → SSH malli test chesi → login successful ga avutundo verify cheyyi.

---

2. EC2 instance is running but the application is not accessible from the internet. What will you check?
Answer:
First, I check whether the application is running and listening on the expected port. Then I verify the Security Group inbound rule, subnet routing, Internet Gateway, and whether the instance has a public IP or is behind a load balancer.
I also test the application locally from the instance to separate an application issue from a network issue.
After fixing the configuration, I test the endpoint from outside the VPC.
Simple ga gurthupettuko:
EC2 up but app access ledu → app port check → Security Group check → public route check → local curl → external curl.
Eng: EC2 is healthy but the application is inaccessible → verify the application port, Security Group, routing, public connectivity, and local response.
Example:
ss -lntp | grep :8080 → Application expected port lo listen chestunda chudu.
curl http://localhost:8080 → Application local ga working aa check cheyyi.
aws ec2 describe-security-groups --group-ids <sg-id> → Application port inbound rule verify cheyyi.
aws ec2 describe-route-tables → Internet Gateway route undho chudu.
curl http://<public-ip>:8080 → External connectivity test cheyyi.
Issue Security Group/route/app binding ayithe → fix chesi → external curl tho application accessible ga undho verify cheyyi.

---

3. EC2 instance is becoming unreachable after deployment. What will you investigate?
Answer:
First, I check the instance status checks and system metrics to determine whether the problem is infrastructure-related or caused by the deployment.
Then I review the deployment changes, CPU, memory, disk, application logs, and network configuration.
I also verify whether the deployment changed firewall rules, services, or system configuration.
If required, I recover the instance or roll back the deployment and then investigate the root cause.
Simple ga gurthupettuko:
Deployment tarvatha EC2 unreachable → instance health check → recent changes → CPU/memory/disk → network/service config → rollback or recover.
Eng: If EC2 becomes unreachable after deployment, compare the deployment changes with instance health and system resources, recover safely, and investigate the root cause.
Example:
aws ec2 describe-instance-status --instance-ids <instance-id> → Instance/system checks chudu.
aws cloudwatch get-metric-statistics ... → CPU metrics inspect cheyyi.
df -h → Disk full ayinda check cheyyi.
free -m → Memory issue unda chudu.
journalctl -xe --no-pager → System errors inspect cheyyi.
git log -5 --oneline → Recent deployment-related code changes chudu.
Issue deployment config ayithe → last known-good version ki rollback/recover chesi → instance connectivity verify cheyyi.

---

4. EC2 CPU suddenly reaches 100%. What will you check?
Answer:
First, I identify which process is consuming the CPU and whether the increase started after a deployment or configuration change.
Then I check application workload, traffic, background processes, and system metrics.
I also verify whether the instance size is sufficient for the workload.
After identifying the cause, I optimize or stop the problematic process and scale the instance if required.
Simple ga gurthupettuko:
CPU 100% → process identify → recent change check → traffic/workload check → optimize or scale.
Eng: High EC2 CPU → identify the process, correlate it with workload and recent changes, then optimize or scale appropriately.

Example:
top → CPU-consuming process identify cheyyi.
ps aux --sort=-%cpu | head → Top CPU processes chudu.
uptime → System load check cheyyi.
aws cloudwatch get-metric-statistics ... → EC2 CPUUtilization trend verify cheyyi.
journalctl -u <service> -n 100 → Application/service errors chudu.
Issue application process ayithe → root cause fix/optimize chesi → CloudWatch CPU normal range ki vachindo verify cheyyi.

---

5. EC2 disk space is full. What will you check?
Answer:
First, I identify which filesystem and directory are consuming the disk. Then I check application logs, temporary files, Docker data, package caches, and old files.
I safely remove unnecessary data and verify whether log rotation and retention are configured correctly.
If the workload genuinely requires more space, I increase the EBS volume and extend the filesystem.
Simple ga gurthupettuko:
Disk full → df chudu → du tho culprit find cheyyi → unnecessary files/logs clean cheyyi → required ayithe EBS expand cheyyi.
Eng: Disk full → identify the filesystem and largest consumers → safely clean data → configure retention → expand EBS when required.
Example:
df -h → Which filesystem full undo chudu.
sudo du -xh /var | sort -h | tail -20 → Large directories identify cheyyi.
sudo journalctl --disk-usage → System logs entha space use chestunnayo chudu.
lsblk → Disk and partition layout check cheyyi.
aws ec2 describe-volumes --volume-ids <volume-id> → EBS volume size verify cheyyi.
Issue unnecessary logs/files ayithe → safe cleanup chesi → df -h tho free space verify cheyyi.

---

6. Application on EC2 cannot connect to an RDS database. What will you check?
Answer:
First, I verify that the RDS instance is available and that the EC2 and RDS networking configuration allows communication.
Then I check the RDS Security Group, EC2 Security Group, subnet routing, database port, credentials, and DNS resolution.
I also test the database port directly from the EC2 instance.
After fixing the network or authentication issue, I verify the application database connection.
Simple ga gurthupettuko:
EC2 → RDS connect kakapothe → RDS status → Security Groups → DB port → DNS → credentials → connectivity test.
Eng: EC2 cannot connect to RDS → check RDS health, Security Groups, routing, database port, DNS, and credentials.
Example:
aws rds describe-db-instances --db-instance-identifier <db> → RDS status and endpoint chudu.
aws ec2 describe-security-groups --group-ids <rds-sg-id> → DB port inbound rule verify cheyyi.
nslookup <rds-endpoint> → RDS DNS resolution test cheyyi.
nc -vz <rds-endpoint> 5432 → PostgreSQL connectivity test cheyyi.
nc -vz <rds-endpoint> 3306 → MySQL connectivity test cheyyi.
Issue SG/network ayithe → fix chesi → DB connection test chesi → application healthy ga undho verify cheyyi.

---

7. RDS database is running, but application queries are very slow. What will you check?
Answer:
First, I check database CPU, memory-related metrics, connections, storage, and latency.
Then I investigate slow queries, missing indexes, connection pool usage, locks, and recent application changes.
I also verify whether the workload has increased suddenly.
Based on the bottleneck, I optimize the query, configuration, or database capacity and monitor the response time again.
Simple ga gurthupettuko:
RDS running but DB slow → CloudWatch metrics → slow queries → connections/locks → workload check → optimize/scale.
Eng: RDS is available but slow → check database metrics, queries, connections, locks, workload, and capacity.
Example:
aws cloudwatch get-metric-statistics ... → RDS CPU/connection/latency metrics check cheyyi.
aws rds describe-db-instances --db-instance-identifier <db> → DB instance configuration verify cheyyi.
Database slow query logs → Slow queries identify cheyyi.
SHOW PROCESSLIST; → MySQL active queries inspect cheyyi.
SELECT * FROM pg_stat_activity; → PostgreSQL active sessions chudu.
Issue query/connection/capacity ayithe → fix chesi → database latency metrics normal ayyaya verify cheyyi.

---

8. Application behind an AWS Load Balancer is returning 502 errors. What will you check?
Answer:
First, I check the target health because a 502 commonly indicates that the load balancer cannot successfully communicate with the backend.
Then I verify target ports, Security Groups, health checks, application listeners, and whether the backend is actually responding.
I also check load balancer and application logs.
After fixing the backend or health-check configuration, I verify that the targets become healthy and 502 errors stop.
Simple ga gurthupettuko:
502 → target health check → port/SG → backend app → LB logs → fix → healthy targets verify.
Eng: AWS Load Balancer 502 → check target health, ports, Security Groups, backend application, and health checks.
Example:
aws elbv2 describe-target-health --target-group-arn <target-group-arn> → Healthy/unhealthy targets chudu.

aws elbv2 describe-target-groups → Target port and health-check configuration verify cheyyi.
ss -lntp → Backend application port listening undho check cheyyi.
curl http://localhost:<port>/health → Backend health endpoint test cheyyi.
aws logs tail <log-group> --follow → Application/LB related logs inspect cheyyi.
Issue target port/health check/app ayithe → fix chesi → aws elbv2 describe-target-health ... → targets healthy ayyaya verify cheyyi.

---

9. AWS Load Balancer is showing healthy targets, but users are getting 504 errors. What will you check?
Answer:
First, I determine whether the backend is responding slowly or timing out.
Then I check application response time, database calls, downstream APIs, load balancer timeout settings, and connection behavior.
I compare the timeout with the application's actual response time.
After identifying the slow dependency, I optimize it or adjust the timeout only when appropriate.
Simple ga gurthupettuko:
504 → target healthy unna response late → app latency → DB/API → timeout settings → bottleneck fix.
Eng: 504 means the target may be healthy but not responding within the expected time, so check application latency and downstream dependencies.
Example:
aws elbv2 describe-load-balancer-attributes --load-balancer-arn <alb-arn> → LB attributes check cheyyi.
curl -w "%{time_total}\n" http://<backend> → Backend response time measure cheyyi.
kubectl logs / application logs → Slow requests identify cheyyi.
iostat → Backend disk latency check cheyyi.
Database monitoring → Slow DB operations check cheyyi.
Issue backend latency ayithe → bottleneck fix chesi → endpoint response time and 504 rate monitor cheyyi.

---

10. AWS Auto Scaling Group is not launching new instances even though CPU is high. What will you check?
Answer:
First, I check whether the scaling policy is configured correctly and whether the CloudWatch alarm is actually triggering.
Then I verify the Auto Scaling Group desired, minimum, and maximum capacity and check whether instance launch is failing because of subnet capacity, quota, AMI, IAM, or launch template issues.
After fixing the scaling configuration or launch problem, I verify that new instances are launched and registered successfully.
Simple ga gurthupettuko:
CPU high but new EC2 launch ledu → scaling alarm → ASG min/max → launch template → subnet/quota/IAM → new instance verify.
Eng: High CPU without scaling → check CloudWatch alarms, scaling policy, ASG limits, launch template, capacity, quotas, and permissions.

1. **Example:**
   aws autoscaling describe-auto-scaling-groups --auto-scaling-group-names <asg> → Desired/min/max capacity chudu.
   aws autoscaling describe-policies --auto-scaling-group-name <asg> → Scaling policy verify cheyyi.
   aws cloudwatch describe-alarms → Scaling alarm state check cheyyi.
   aws autoscaling describe-scaling-activities --auto-scaling-group-name <asg> → Launch failure reason chudu.
   aws ec2 describe-launch-template-versions --launch-template-id <id> → Launch configuration verify cheyyi.
   Issue scaling policy/launch configuration ayithe → fix chesi → ASG activity lo successful launch verify cheyyi.

   11. EC2 instance has no internet access, but the instance is running. What will you check?

Answer:
First, I check whether the instance is in a public or private subnet and whether it has the required public IP or NAT connectivity.
Then I verify the route table, Internet Gateway or NAT Gateway, Security Group, and Network ACL.
I also test DNS resolution and external connectivity from the instance.
After identifying the routing or network issue, I fix it and verify internet access again.
Simple ga gurthupettuko:
EC2 ki internet lekapothe → public/private subnet check → route table → IGW/NAT → SG/NACL → DNS check → connectivity verify.
Eng: If EC2 has no internet → check subnet type, routes, IGW/NAT, Security Group, NACL, and DNS → fix and retest.
Example:
aws ec2 describe-instances --instance-ids <instance-id> → Public IP, subnet, and VPC details chudu.
aws ec2 describe-route-tables → 0.0.0.0/0 route correct ga undho check cheyyi.
aws ec2 describe-internet-gateways → Internet Gateway VPC ki attached undho verify cheyyi.
aws ec2 describe-nat-gateways → Private subnet ki NAT Gateway available ga undho chudu.
ip route → Instance routing table check cheyyi.
nslookup google.com → DNS resolution test cheyyi.
curl -I https://google.com → Internet connectivity test cheyyi.
Issue route/IGW/NAT ayithe → fix chesi → curl tho internet access successful ga undho verify cheyyi.

---

12. Private subnet instances cannot download packages from the internet. What will you check?
Answer:
First, I verify that the instances are actually in a private subnet and that the private subnet route table points internet-bound traffic to a NAT Gateway.
Then I verify that the NAT Gateway is available and located in a subnet with a working route to an Internet Gateway.
I also check Network ACLs, DNS, and the package repository connectivity.
After fixing the NAT or routing configuration, I test package download again.

Simple ga gurthupettuko:
Private subnet package download fail → NAT Gateway → private route table → NAT public route → NACL/DNS → package download test.
Eng: Private instances need NAT for outbound internet access, so verify the NAT Gateway, routes, NACLs, DNS, and repository connectivity.
Example:
aws ec2 describe-route-tables → Private subnet route table lo 0.0.0.0/0 NAT Gateway ki point avutunda chudu.
aws ec2 describe-nat-gateways → NAT Gateway state available lo undho check cheyyi.
aws ec2 describe-route-tables --filters Name=route.nat-gateway-id,Values=<nat-id> → NAT routing verify cheyyi.
curl -I https://<package-repository> → Repository reachability test cheyyi.
sudo yum update / sudo apt update → Package download test cheyyi.
Issue NAT/route ayithe → fix chesi → package installation successful ga avutunda verify cheyyi.

---

13. Two EC2 instances in the same VPC cannot communicate with each other. What will you check?
Answer:
First, I verify that both instances are in the expected VPC and that their private IP addresses are reachable.
Then I check the Security Groups attached to both instances, Network ACLs, subnet routes, and the application listening port.
I also verify whether host-level firewall rules are blocking the traffic.
After correcting the network or firewall configuration, I test connectivity again.
Simple ga gurthupettuko:
EC2-to-EC2 communication fail → same VPC/subnet check → SG → NACL → route → OS firewall → port test.
Eng: If EC2 instances cannot communicate, verify VPC/subnets, Security Groups, NACLs, routes, host firewall, and application ports.
Example:
aws ec2 describe-instances --instance-ids <id1> <id2> → VPC, subnet, and private IP compare cheyyi.
aws ec2 describe-security-groups --group-ids <sg-id> → Source SG/IP ki required port allowed undho check cheyyi.
aws ec2 describe-network-acls → Subnet traffic rules verify cheyyi.
ping <private-ip> → Basic connectivity test cheyyi.
nc -vz <private-ip> 8080 → Application port connectivity test cheyyi.
sudo iptables -L -n → OS firewall traffic block chestunda chudu.
Issue SG/NACL/firewall ayithe → fix chesi → nc tho communication successful ga undho verify cheyyi.

---

14. Security Group rule looks correct, but traffic is still blocked. What will you investigate?
Answer:
First, I verify that the correct Security Group is actually attached to the target resource.
Then I check whether the traffic is being blocked by a Network ACL, incorrect route, load balancer configuration, or host firewall.
I also verify the source IP or source Security Group and the exact destination port.
After identifying the blocking layer, I correct it and retest connectivity.
Simple ga gurthupettuko:
SG correct unna traffic block → correct SG attached aa → NACL → route → source/destination port → OS firewall check.
Eng: If the Security Group looks correct but traffic fails, check the attached SG, NACL, routing, source, destination port, and host firewall.
Example:
aws ec2 describe-instances --instance-ids <instance-id> → Correct Security Group attached undho verify cheyyi.
aws ec2 describe-security-groups --group-ids <sg-id> → Inbound/outbound rules chudu.
aws ec2 describe-network-acls → NACL deny rule undho check cheyyi.
ip route → Correct network route undho verify cheyyi.
sudo ss -lntp → Application expected port lo listening undho chudu.
nc -vz <host> <port> → End-to-end connectivity test cheyyi.
Issue NACL/route/host firewall ayithe → fix chesi → connectivity malli verify cheyyi.

---

15. An application suddenly gets AccessDenied when accessing an S3 bucket. What will you check?
Answer:
First, I identify which IAM identity the application is using and verify its attached policies.
Then I check the S3 bucket policy, object permissions, encryption configuration, and whether an explicit deny exists.
I also verify that the application is accessing the correct bucket and object path.
After fixing the permission issue, I test the same S3 operation again.
Simple ga gurthupettuko:
S3 AccessDenied → IAM identity check → IAM policy → bucket policy → explicit deny → KMS/path check → retry.
Eng: S3 AccessDenied → identify the IAM identity, check IAM and bucket policies, explicit denies, encryption, and object path.
Example:
aws sts get-caller-identity → Application/user actually ye AWS identity use chestundo chudu.
aws iam list-attached-role-policies --role-name <role> → Role policies check cheyyi.
aws s3api get-bucket-policy --bucket <bucket> → Bucket policy inspect cheyyi.
aws s3api head-object --bucket <bucket> --key <key> → Object access test cheyyi.
aws s3 ls s3://<bucket> → Bucket access verify cheyyi.
Issue IAM/bucket policy ayithe → required minimum permission add chesi → S3 operation successful ga avutunda verify cheyyi.

---

16. S3 upload succeeds, but the application cannot read the uploaded object. What will you check?
Answer:
First, I verify that the object was actually uploaded to the expected bucket and key.

Then I check whether the upload and read operations are using the same or different IAM identities.
I also check bucket policies, object ownership, encryption, KMS permissions, and the exact object path.
After correcting the read permission or encryption access, I test object retrieval again.
Simple ga gurthupettuko:
Upload success but read fail → object path check → IAM identity compare → bucket/object permission → KMS check → download test.
Eng: If S3 upload works but read fails, verify the object path, IAM identities, bucket/object permissions, ownership, and KMS access.
Example:
aws s3api head-object --bucket <bucket> --key <key> → Object exists aa check cheyyi.
aws sts get-caller-identity → Current IAM identity chudu.
aws s3api get-object --bucket <bucket> --key <key> /tmp/test-file → Read access test cheyyi.
aws s3api get-bucket-encryption --bucket <bucket> → Encryption configuration check cheyyi.
aws kms get-key-policy --key-id <key-id> → KMS permissions verify cheyyi.
Issue read/KMS permission ayithe → required access fix chesi → object successfully download avutunda verify cheyyi.

---

17. An IAM user or role has Administrator-like permissions, but an AWS action is still denied. Why?
Answer:
First, I check whether there is an explicit deny because an explicit deny takes precedence over an allow.
Then I check Service Control Policies, permissions boundaries, session policies, resource-based policies, and identity policies.
I also verify whether the request is being made in the correct account, region, or resource.
After identifying the restricting policy, I correct it and test the action again.
Simple ga gurthupettuko:
Permission undi but AccessDenied → explicit deny first → SCP → permissions boundary → session/resource policy → account/region check.
Eng: Even broad permissions can be denied by an explicit deny, SCP, permissions boundary, session policy, or resource policy.
Example:
aws sts get-caller-identity → Correct account and identity aa chudu.
aws iam get-role --role-name <role> → Role details verify cheyyi.
aws iam list-role-policies --role-name <role> → Inline policies inspect cheyyi.
aws iam list-attached-role-policies --role-name <role> → Attached policies chudu.
AWS Policy Simulator → Specific action allowed/denied reason test cheyyi.
AWS Organizations → SCP restrictions verify cheyyi.
Issue explicit deny/SCP/boundary ayithe → policy restriction fix chesi → AWS action malli test cheyyi.

---

18. EBS volume is attached to EC2, but the application cannot use the disk. What will you check?
Answer:
First, I verify that the EBS volume is attached and available to the expected instance.
Then I check whether the device is visible to the operating system, whether it has a filesystem, whether it is mounted, and whether there is enough free space.
I also check filesystem permissions and application configuration.
After mounting or fixing the filesystem issue, I verify that the application can write to the disk.
Simple ga gurthupettuko:
EBS attached but app use cheyyalekapothe → AWS attachment → OS device → filesystem → mount → permissions → space check.
Eng: EBS attached but unusable → verify attachment, OS device, filesystem, mount point, permissions, and disk space.
Example:
aws ec2 describe-volumes --volume-ids <volume-id> → Volume state and attachment check cheyyi.
lsblk → OS lo EBS device kanipistunda chudu.
sudo blkid → Filesystem undho verify cheyyi.
df -h → Mount and free space check cheyyi.
mount | grep <device> → Volume mounted undho chudu.
sudo mount <device> <mount-point> → Required ayithe mount cheyyi.
Issue mount/filesystem ayithe → fix chesi → application write operation successful ga undho verify cheyyi.

---

19. EC2 instance suddenly has very high network traffic. What will you investigate?
Answer:
First, I check inbound and outbound network metrics to determine the direction of the traffic.
Then I identify which process or application is generating the traffic and correlate it with recent deployments, traffic increases, or unexpected activity.
I also check load balancer traffic and application logs.
If the traffic is abnormal, I contain the issue safely and investigate the source before restoring normal operation.
Simple ga gurthupettuko:
Network traffic sudden high → inbound/outbound identify → process find → app/LB logs → recent changes → abnormal ayithe contain and fix.
Eng: High EC2 network traffic → determine traffic direction, identify the process, check application/LB logs and recent changes, then contain abnormal activity.
Example:
aws cloudwatch get-metric-statistics ... → NetworkIn/NetworkOut trend chudu.
ss -tunap → Active network connections identify cheyyi.
sudo lsof -i -n -P → Which process network use chestundo chudu.
top → Resource-consuming processes verify cheyyi.
Application logs → Unexpected requests or traffic source identify cheyyi.
Issue unwanted process/traffic ayithe → safely isolate/fix chesi → CloudWatch metrics normal range ki vachaya verify cheyyi.

20. Route 53 DNS is pointing to the correct resource, but users still cannot access the application. What will you check?
Answer:
First, I verify that DNS is resolving to the expected record and endpoint.
Then I check whether the load balancer, application, or target behind that record is healthy.
I also verify TTL, DNS propagation, HTTPS certificate configuration, and whether the endpoint is reachable from different networks.
After identifying whether the issue is DNS, certificate, or application-side, I fix it and verify the complete request path.
Simple ga gurthupettuko:
DNS correct unna app access ledu → DNS resolve check → LB/app health → certificate → TTL/propagation → endpoint test.
Eng: If Route 53 resolves correctly but the application is inaccessible, verify DNS resolution, backend health, certificates, propagation, and endpoint connectivity.
Example:
aws route53 list-resource-record-sets --hosted-zone-id <zone-id> → DNS record correct ga undho chudu.
dig +short app.example.com → DNS IP/hostname resolution verify cheyyi.
nslookup app.example.com → DNS resolution cross-check cheyyi.
curl -Iv https://app.example.com → DNS + HTTPS connectivity test cheyyi.
openssl s_client -connect app.example.com:443 -servername app.example.com → TLS certificate issue undho chudu.
aws elbv2 describe-target-health --target-group-arn <target-group-arn> → Backend healthy aa verify cheyyi.
Issue DNS/certificate/backend ayithe → fix chesi → curl -Iv tho complete application access verify cheyyi.